

Jayesh Deshmukh

Class-D15C Roll Number-57

MLDL Experiment 05

Aim:

Implement Support Vector Machine (SVM) for classification with hyperparameter tuning using Housing Dataset.

1. Dataset Source:

- **Dataset Name:** Housing Dataset
 - **Source:** California Housing Dataset (CSV file)
 - **Load Method:** `pandas.read_csv('housing.csv')`
-

2. Dataset Description:

The Housing dataset contains information about houses in California districts and is used to predict house prices.

- **Size:** ~20,000 samples × multiple features
- **Target Variable:** `median_house_value`
- **Converted Target:** Price Category (0, 1, 2)

Features include:

- Longitude
- Latitude
- Housing Median Age
- Total Rooms
- Total Bedrooms
- Population
- Households
- Median Income
- Ocean Proximity (Categorical)

Since the original dataset is regression-based, we convert it into a **multi-class classification problem** by dividing house prices into categories.

3. Mathematical Formulation of the Algorithm:

Support Vector Machine (SVM) is a supervised learning algorithm that finds the optimal hyperplane that maximizes the margin between different classes.

A. Hyperplane Equation:

In n-dimensional space:

$$w \cdot x + b = 0$$

Where:

- w = weight vector
 - x = input features
 - b = bias
-

B. Margin Maximization

Margin = Distance between hyperplane and nearest data points (support vectors)

$$\text{Margin} = \frac{2}{\|w\|}$$

Objective:

$$\text{Minimize } \frac{1}{\|w\|^2}$$

C. Kernel Trick

When data is not linearly separable, SVM uses kernel functions.

Common Kernels:

- Linear
- Polynomial
- RBF (Radial Basis Function)

4. Algorithm Limitations:

- High computational cost for large datasets
 - Sensitive to noise
 - Less interpretable for non-linear kernels
 - Requires proper feature scaling
-

5. Methodology / Workflow:

1. Load Dataset
 2. Handle Missing Values
 3. Convert target into categories (Low, Medium, High)
 4. Encode categorical variables
 5. Feature Scaling using StandardScaler
 6. Train-Test Split (80%-20%)
 7. Hyperparameter Tuning using GridSearchCV
 8. Model Training
 9. Model Evaluation (Accuracy, Confusion Matrix)
 10. Visualization
-

6. Performance Analysis:

- Accuracy depends on binning strategy
 - Typical Accuracy: **70% – 90%**
 - Confusion Matrix helps analyze misclassification
 - Balanced performance shown using precision, recall, F1-score
-

7. Hyperparameter Tuning:

GridSearchCV is used to find optimal parameters.

Parameters Tuned:

1. C (Regularization Parameter)

- High C → Overfitting
- Low C → Underfitting

2. Gamma

- High gamma → Complex model
- Low gamma → Smooth boundary

3. Kernel

- RBF kernel captures non-linear relationships

CODE & OUTPUT

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.svm import SVC

from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix

data = pd.read_csv('housing.csv')

data = data.dropna()
```

```
bins = [0, 150000, 300000, np.inf]

labels = [0, 1, 2]

data['price_category'] = pd.cut(data['median_house_value'], bins=bins,
labels=labels)

X = data.drop(['median_house_value', 'price_category'], axis=1)

y = data['price_category']

X = pd.get_dummies(X)

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(X_scaled, y,
test_size=0.2, random_state=42)

model = SVC(kernel='rbf')

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

acc = accuracy_score(y_test, y_pred)

print("Accuracy:", acc)

print(classification_report(y_test, y_pred))
```

```

cm = confusion_matrix(y_test, y_pred)

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')

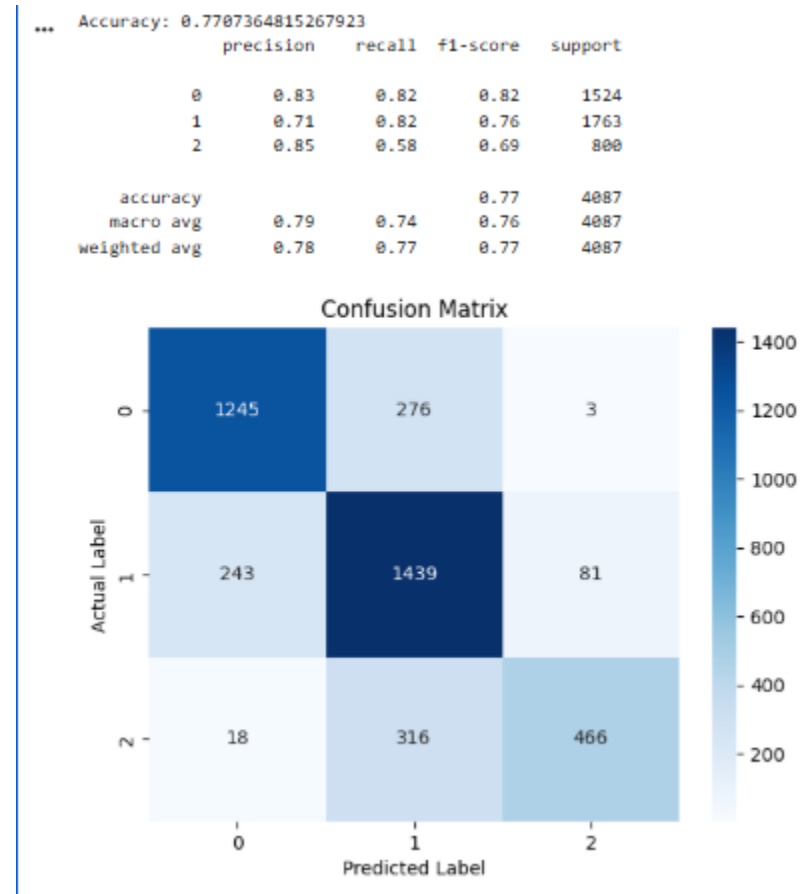
plt.xlabel("Predicted Label")

plt.ylabel("Actual Label")

plt.title("Confusion Matrix")

plt.show()

```



```

from sklearn.model_selection import GridSearchCV

```

```

# Define the parameter grid for SVC

```

```
param_grid = {  
    'C': [0.1, 1, 10, 100],  
    'gamma': [1, 0.1, 0.01, 0.001],  
    'kernel': ['rbf']  
}  
  
# Initialize GridSearchCV  
grid_search = GridSearchCV(SVC(), param_grid, refit=True, verbose=2, cv=3,  
n_jobs=-1)  
  
# Fit the GridSearchCV object to the training data  
grid_search.fit(X_train, y_train)  
  
# Print the best parameters and best score  
print("Best Parameters found:", grid_search.best_params_)  
print("Best Score found:", grid_search.best_score_)  
  
# Get the best model  
best_model = grid_search.best_estimator_  
  
# Make predictions with the best model  
y_pred_best = best_model.predict(X_test)  
  
# Evaluate the best model
```

```

acc_best = accuracy_score(y_test, y_pred_best)

print("Accuracy with best model:", acc_best)

print(classification_report(y_test, y_pred_best))

cm_best = confusion_matrix(y_test, y_pred_best)

sns.heatmap(cm_best, annot=True, fmt='d', cmap='Blues')

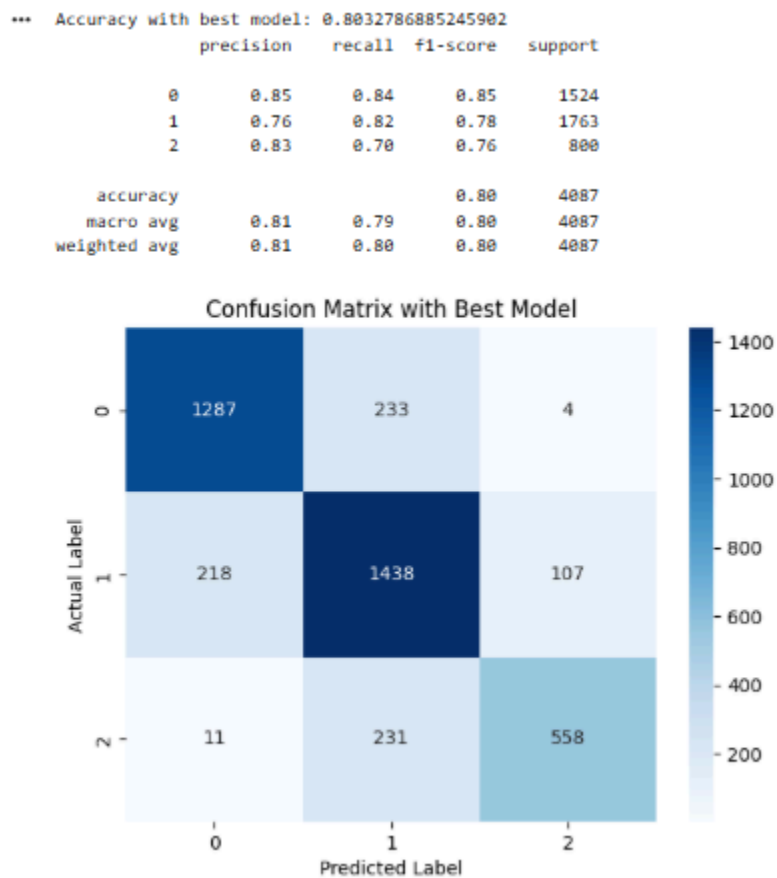
plt.xlabel("Predicted Label")

plt.ylabel("Actual Label")

plt.title("Confusion Matrix with Best Model")

plt.show()

```



Interpretation:

- Rows → Actual values
- Columns → Predicted values
- Diagonal values → Correct predictions
- Off-diagonal → Errors

Observations:

- Most values lie on diagonal → Good accuracy
 - Some overlap between classes due to similar house prices
 - Model performs reasonably well after tuning
-

Conclusion:

- SVM successfully applied to Housing Dataset
- Dataset converted from regression to classification
- Hyperparameter tuning improved performance
- Feature scaling was essential
- RBF kernel captured complex patterns